● SATOSHI CLOUD MINER · POSTMORTEM · v2

Foolproof iOS Clone Prompts Playbook

Copy/paste prompts that ship an Expo + FastAPI iOS app to TestFlight with zero rebuild loops. Includes a Litecoin + Dogecoin variant.

HOW TO USE

- You are about to clone an iOS app and ship it to TestFlight.
- You don't want a 20-build, 20-credit rebuild loop.
- You want to read a few pages, copy, paste, walk away.
- This playbook is built for exactly that.

BASED ON Builds #11 → #20 of Satoshi Cloud Miner. We made every mistake so you do not have to. The prompts here would have collapsed that journey into 1–2 EAS builds.

WHAT IS NEW IN v2 (1) All tables re-rendered with word-wrap so no text overlaps another column. (2) New Section 6 — a complete sub-playbook for cloning the same architecture for Litecoin + Dogecoin (one app, both coins).

0 · QUICKSTART

How to use this playbook

This document contains **10 numbered prompts** + **1 master prompt** for the Bitcoin / Satoshi Cloud Miner case, plus a full **Litecoin + Dogecoin variant** in Section 6. If you follow them in order, you will skip every painful rebuild loop we hit (App Store Connect IAP mismatches, icon white-borders, simulated data, iPad screenshot rejections, tracking-permission rejections, etc.).

The three rules

1 · Send the Master Prompt first.	It declares every constraint upfront. Without it, the agent invents assumptions that cost you credits.
2 · Send each follow-up prompt in order.	Each one assumes the previous step is done. Do not skip ahead — they are sequential.
3 · Replace every <…> placeholder.	Wherever you see angle brackets, paste your real value. If you cannot fill one, ask the agent to source it for you in the same chat.

Before you paste anything — collect these credentials

Item	Where to get it	Placeholder you will replace
Apple Team ID	developer.apple.com → Membership	`<TEAM_ID>`
ASC App Manager API key (.p8)	App Store Connect → Users & Access → Integrations → App Store Connect API	`<ASC_KEY_ID>` / `<ASC_ISSUER_ID>` / `<ASC_P8>`
In-App Purchase Server API key (.p8)	App Store Connect → Users & Access → Integrations → In-App Purchase	`<IAP_KEY_ID>` / `<IAP_P8>`
Expo personal access token	expo.dev → account settings → access tokens	`<EXPO_TOKEN>`
Existing ASC app record	App Store Connect → Apps	`<BUNDLE_ID>` / `<ASC_APP_ID>`
LLM key	Emergent universal LLM key (recommended), or your OpenAI / Anthropic / Gemini key	`<LLM_KEY>`
Payment / wallet keys	Whatever vendor (Stripe, Blink, BTCPay, NowPayments, Twilio…)	`<VENDOR_KEY>`

1 · THE MASTER PROMPT

Send this one prompt FIRST. Always.

This is the single most important message in the whole playbook. Send it as the very first message in a fresh chat. Every gotcha that burned credits on Satoshi Cloud Miner is preempted by a constraint in this prompt.

WHY: Without this prompt, the agent will assume sensible defaults that do not match Apple's reality (iPad screenshots required, hardcoded BTC rate, AI-generated icon with white border, etc.).

■ PROMPT 00 THE MASTER PROMPT

GOAL

Clone <SOURCE APP NAME + URL OR SCREENSHOTS> into a production-ready iOS app called <MY APP NAME> and ship it to TestFlight + the App Store. I will NOT review interim builds — deliver the final result. You decide every implementation detail; I only weigh in on product behavior or branding.

MY CREDENTIALS (use these, do not ask later)

- Apple Team ID: <TEAM_ID>
- ASC API key: <ASC_KEY_ID> / <ASC_ISSUER_ID> (.p8 attached)
- IAP Server API key: <IAP_KEY_ID> (.p8 attached)
- EXPO_TOKEN: <EXPO_TOKEN>
- ASC app record: bundle=<BUNDLE_ID> app_id=<ASC_APP_ID>
- LLM key: Use the Emergent universal LLM key
- Payment/wallet keys: <VENDOR_KEYS_HERE>

NON-NEGOTIABLE CONSTRAINTS — fail the build if any is violated

1. No simulated data. No hardcoded prices, fake rates, deterministic-random AI, or MOCK_* constants. If a real source is unavailable, surface 'unavailable' in the UI rather than fake it.
2. App Store Connect is the SOURCE OF TRUTH for IAP product IDs. Enumerate ASC via API BEFORE writing any IAP code. Never invent a product_id in the backend that doesn't already exist in ASC.
3. App icon: 1024x1024 PNG, RGB (no alpha), full-bleed dark background, no white border, no transparent corners. Programmatically sample edge pixels before committing.
4. AI-generated assets must have correct magic bytes. If filename says .png, file MUST start with 89 50 4E 47.
5. supportsTablet: false by default (otherwise Apple requires iPad screenshots).
6. Do NOT declare iOS permissions you don't use. No NSUserTrackingUsageDescription unless an SDK actually requests tracking — otherwise Apple forces the App Privacy questionnaire.
7. Quality gates before EVERY EAS build: npx expo-doctor (must be N/N), yarn tsc --noEmit, backend tests. Block the build on any failure.
8. Use the App Store Connect API to upload everything — description, keywords, screenshots, preview video, IAP linkage, review submission. Do NOT write me a 'manual upload guide'.

DEFINITION OF DONE

- App Store Connect shows WAITING_FOR_REVIEW with all IAPs auto-bundled.
- Backend regression tests pass.
- A persistent auto_ship watcher is armed for v1.0.1 so I never need to manually ship a follow-up build.

Confirm you understand every constraint before writing code. Push back on anything that needs Apple policy exceptions (gambling, crypto withdrawals, age-gated content). Then proceed end-to-end.

2 · THE 7 GOTCHAS

Failures we hit — and the one sentence that would have prevented each

Add all seven lines to the bottom of the Master Prompt if you want belt-and-suspenders coverage.

#	What went wrong on Satoshi Cloud Miner	Sentence to add to your prompt
1	starter_099 IAP existed in backend code but Apple never had it. Every TestFlight tap returned 'Purchase failed'.	Before writing any IAP code, enumerate App Store Connect IAPs via the v2 API and use those product IDs as the source of truth.
2	BTC_USD_RATE = 65000.0 was hardcoded as a 'simulated rate' and ran in production for weeks.	Any price, FX, or rate must come from a live API with a fallback cascade. No magic numbers outside true constants like SATS_PER_BTC.
3	AI Trading Agents were deterministic random — the UI said 'AI' but the data was random.choice().	Anywhere the UI shows an 'AI' or 'live' value, it must be a real LLM/API call with cache and offline fallback. No fake AI.
4	AI-generated icon was JPEG bytes saved as .png, which broke the EAS expo-doctor check.	For every generated image, verify the file's magic bytes match the extension. Re-encode through PIL if mismatched.
5	Icon had a 69-px white border baked in, which produced a visible white ring around the app on the home screen.	Sample the four edges of any 1024×1024 icon. If pixels above (235,235,235) are found within 80 px of the edge, replace with the brand background.
6	supportsTablet:true plus an unused NSUserTrackingUsageDescription caused submission rejection.	Default supportsTablet:false. Never add a permission string unless I explicitly request the feature.
7	Apple's 'first IAP must ship with first version' plus the screenshot lockout once a version enters WAITING_FOR_REVIEW caught us mid-flow.	Use the reviewSubmissions API (not inAppPurchaseSubmissions). Upload all metadata and screenshots BEFORE attaching the version to the review submission.

3 · THE 10 NUMBERED PROMPTS

Send these in order, one per message

Each prompt assumes the previous one finished cleanly. Wait for the agent to confirm success (a green log line, a screenshot, or an HTTP 200 response) before sending the next one.

■ PROMPT 01 Lock in the product spec

```
Here is my PRODUCT_SPEC.md (attached). Read every section and confirm
you understand it. Specifically confirm:
  - the exact list of screens
  - every IAP tier with product_id, type, price
  - every external integration with key scopes
  - every iOS permission with the code path that uses it

Push back on anything that requires an Apple App Store policy
exception. Propose alternatives where needed. Reply with a TL;DR
of the spec in your own words so I know we are aligned.
```

■ PROMPT 02 Build the backend + Expo skeleton (no UI yet)

```
Build the FastAPI backend + Expo Router skeleton.
  - Auth: email+password + JWT, with /api/auth/{register,login,me}.
  - All data models from the spec, with MongoDB indexes.
  - Every API endpoint stubbed with real schema responses (200 OK).
  - expo-router wired with app/(tabs)/_layout.tsx, but tabs
    render placeholder Text only — no real UI yet.

Then run the deep_testing_backend_v2 agent and paste the report.
Do NOT proceed to UI until backend regression is green.
```

■ PROMPT 03 Bootstrap App Store Connect (BEFORE any IAP code)

```
Using my ASC App Manager key, do this end-to-end via the ASC API:
  1. Verify the existing app record <ASC_APP_ID>.
  2. Create every IAP from PRODUCT_SPEC.md with the correct type
     (consumable / non-consumable / subscription), localized name,
     price tier, and review screenshot (use a 1290x2796 placeholder
     PNG if needed).
  3. Move each IAP to READY_TO_SUBMIT state.

After running, print the FINAL list of product IDs that exist in ASC.
These are the source of truth for the backend SHOP_PACKAGES table.
If any spec IAP cannot be created via API (e.g. missing review screenshot),
tell me explicitly so we remove it from the spec.
```

■ PROMPT 04 Wire integrations one at a time, prove each is live

```
For each integration in my spec, do this loop:
  1. Write the integration module (one file in backend/integrations/).
  2. Add a /api/diag/<integration_name> endpoint that performs ONE
     real round-trip to the vendor (e.g. fetch BTC rate, validate
     a test Apple receipt, send a $0.001 Lightning ping).
  3. Hit it and paste the JSON response in your reply.

Do NOT move to the next integration until the diag endpoint returns
a real, non-mocked response. No 'MOCKED' values anywhere.
```

■ PROMPT 05 Build the UI from the spec

```
Build every screen from PRODUCT_SPEC.md.
  - After EVERY screen, run yarn tsc --noEmit and npx expo-doctor.
  - Capture a screenshot of the rendered screen from the web preview
    (http://localhost:3000) at iPhone 14 Pro dimensions and paste it.
  - Use real data from the integrations wired in Prompt 04 — no
    placeholder text, no Lorem Ipsum, no MOCK_*.

If any check fails, stop and fix that screen before moving on.
Do not pile up TODOs.
```

■ PROMPT 06 Generate marketing assets — with pixel-level verification

```
Generate all App Store marketing assets in ONE batch:
  - App icon: 1024x1024 PNG, RGB only (no alpha), brand-colored
    full-bleed background, glyph centered, NO white border.
  - 12 screenshots: 1290x2796 (iPhone 6.7), 1242x2688 (6.5), 1242x2208 (5.5).
  - 1 App Preview video: 1080x1920 H.264, 15-30s, STEREO or SILENT
    audio only (no mono — Apple's encoder rejects MOV_RESAVE_STEREO).

Then run a verification pass that PROVES:
  - icon edge pixels (sample 10 corners within 20 px of edge) are NOT white
  - every PNG starts with bytes 89 50 4E 47
  - video probe (ffprobe) shows acodec=aac stereo OR no audio stream

Paste the pixel-sample report. Re-encode if anything fails.
```

■ PROMPT 07 Harden app.json for Apple submission

```
Update /app/frontend/app.json with these EXACT settings before any
EAS build:
  - ios.supportsTablet: false
  - ios.config.usesNonExemptEncryption: false
  - ios.infoPlist.ITSAppUsesNonExemptEncryption: false
  - Remove every permission string (NSCameraUsageDescription,
    NSUserTrackingUsageDescription, etc.) UNLESS my spec lists a
    feature that actually requires it.
  - version: 1.0.0 buildNumber: 1

Then run npx expo-doctor — must show N/N checks passed.
```

■ PROMPT 08 Ship to App Store Connect end-to-end (one shot)

```
Now run the full ship sequence WITHOUT pausing for input:
  1. npx eas build --platform ios --profile production --non-interactive
     Wait for it to finish.
  2. npx eas submit --platform ios --non-interactive (uses my ASC key).
  3. Poll the ASC builds API until the build state is VALID.
  4. Attach the build to App Store Version 1.0 via the ASC API.
  5. Upload description / keywords / supportURL / marketingURL /
     promotionalText via the appStoreVersionLocalizations PATCH.
  6. Upload the 12 screenshots + the App Preview video.
  7. Create a reviewSubmission, attach the version as a
     reviewSubmissionItem (IAPs auto-bundle), PATCH submitted:true.

Print the final reviewSubmission state. If anything fails, surface
Apple's exact error message — do NOT retry blindly.
```

■ PROMPT 09 Arm the auto_ship watcher for v1.0.1+

```
Create /app/backend/services/auto_ship.py with these properties:
  - Runs every 30 min via APScheduler inside the FastAPI worker.
  - Polls the ASC API for version 1.0's appStoreState.
  - When the state becomes READY_FOR_SALE or PENDING_APPLE_RELEASE,
    it auto-ships the next version: bump app.json, run eas build,
    eas submit, attach to version 1.0.1, upload fresh screenshots,
    submit reviewSubmission.
  - Idempotent: stores state in /app/store/.auto_ship_state.json so
    it never double-ships.

Confirm the watcher's first tick logged successfully and paste the log
line that proves it.
```

■ PROMPT 10 Final smoke test + hand-off

```
Run a final regression pass:
  - deep_testing_backend_v2 (all critical endpoints).
  - Take a fresh web preview screenshot of every tab.
  - curl GET /api/system/btc_rate (or your live-data endpoint) and
    paste the JSON to prove no simulated data is left.
  - Show the ASC final state: version, build, IAP states,
    reviewSubmission.

Then write a /app/HANDOFF.md that documents:
  - every credential the app uses + where it lives
  - every cron job + its schedule
  - how to invoke the auto_ship watcher manually if needed
I want to be able to hand this to ANY future engineer in one file.
```

4 · IF YOU ONLY READ TWO LINES

The two sentences that would have saved 70% of the credits on Satoshi Cloud Miner

If you can only send two sentences before letting the agent work, these are the two.

SENTENCE 1

"Before writing a single line of IAP code, enumerate my App Store Connect product IDs via API and tell me the canonical list. I will only use those — no new SKUs invented in the backend."

SENTENCE 2

"Any 'AI', 'rate', 'price', or 'live data' field in the UI MUST be backed by a real provider with a fallback cascade. Show me one curl response per data source proving the wire is live before connecting it to the UI."

The fool-proof header — paste it as the first 8 lines of any iOS-clone prompt

■ PROMPT HEADER **FOOL-PROOF HEADER**

```
ROLE: ship-the-app engineer, not interim-demo engineer.
DEFINITION OF DONE: WAITING_FOR_REVIEW in App Store Connect.
CREDS: <paste all keys here, do not ask me later>
HARD CONSTRAINTS:
  - Zero simulated data.
  - App Store Connect is the source of truth for product IDs.
  - Full-bleed RGB icon, no alpha, no white border, pixel-verified.
  - supportsTablet:false unless explicitly requested.
  - No unused iOS permission strings.
  - expo-doctor + tsc must pass before any EAS build.
  - Use the ASC API for every upload; no manual upload guides.
EXIT CRITERIA: auto_ship watcher armed for the next version.
```

5 · TL;DR — THE ONE LESSON

Verify the platform's reality BEFORE writing the code that depends on it

Both of the most expensive bugs on Satoshi Cloud Miner — the **starter_099 'Purchase failed'** in TestFlight and the **white-border app icon** — came from the same mistake: writing code based on what we *assumed* the platform or asset-generator had produced, instead of programmatically inspecting what was actually there.

If every future iOS clone starts with a 5-minute *'enumerate the platform reality'* step (list ASC's actual IAPs, sample the icon's actual edge pixels, probe the actual video's audio track), you ship in 1–2 EAS builds. Not 20.

The four reality-checks every iOS clone needs on Day 1

#	Reality check	How to do it (one line)
1	What product IDs exist in App Store Connect?	`GET /v1/apps/<id>/inAppPurchasesV2 — list and pin those as your SKU table.`
2	What does the 1024x1024 icon actually look like at the edges?	`PIL: sample 10 random pixels within 30 px of each edge. Assert non-white.`
3	What is the live BTC / FX / whatever rate right now?	`curl the provider. Cache. Add two fallback providers. No constants in code.`
4	What audio track does my preview video really have?	`ffprobe -v error -show_streams app-preview.mp4 | grep audio — assert stereo or none.`

6 · LITECOIN + DOGECOIN VARIANT

Cloning the architecture for Litecoin and Dogecoin (one app, both coins)

This is a complete sub-playbook for building a brand-new iOS app — let's call it **<MY_APP_NAME>** — that uses the same architecture as Satoshi Cloud Miner, but supports Litecoin (LTC) and Dogecoin (DOGE) instead of Bitcoin. One app, both coins, with a coin selector in the wallet UI.

> **HOW:** This sub-playbook re-uses every constraint from Section 1's Master Prompt. Send **that** first (with the LTC+DOGE variant on the next page substituted for <GOAL>). Then send Prompts L01–L10 from Section 6 in order.

What is different from Satoshi Cloud Miner

Area	Satoshi Cloud Miner (BTC)	<MY_APP_NAME> (LTC + DOGE)
Coins supported	One — Bitcoin only.	Two — Litecoin AND Dogecoin in a single app, with a coin-picker in the wallet UI.
Withdrawal method	Lightning Network via Blink Wallet (Bitcoin Layer-2).	On-chain payouts via NowPayments REST API (preferred) — supports both LTC and DOGE with one account. Alternatives noted below.
Live price feed	1 ticker (BTC/USD) from CoinGecko → Coinbase → Kraken.	2 tickers (LTC/USD + DOGE/USD) using the same 3-provider cascade pattern. Both cached, 5-min refresh.
AI Trading Agents	6 agents commenting on BTC mining + Lightning conditions.	Same 6 agents, but each daily commentary must reference whichever coin the user is currently viewing (selector-aware).
IAP price tiers	10 SKUs: welcome_199 → colossus_19999 + adfree_399 (USD).	Identical USD price ladder — Apple still charges in USD. Plan yields are quoted in LTC AND DOGE per plan, the user picks.
Apple receipt validation	App Store Server API with In-App Purchase .p8 key.	Identical. No change.
Branding	Dark theme + neon green.	Dark theme + dual accent: Litecoin silver-blue (#345D9D) + Dogecoin gold (#C2A633). App icon shows both glyphs over a dark background.

Extra credentials YOU need to collect for the LTC + DOGE variant

Item	Where to get it	Placeholder you will replace
NowPayments API key	nowpayments.io → Dashboard → Settings → API keys	`<NOWPAY_API_KEY>`
NowPayments IPN secret	nowpayments.io → Settings → IPN (for payout webhook verification)	`<NOWPAY_IPN_SECRET>`
Custodial LTC payout address (optional)	Generate in any LTC wallet you control (Litecoin Core, Exodus, etc.)	`<LTC_PAYOUT_ADDR>`
Custodial DOGE payout address (optional)	Generate in any DOGE wallet you control (Dogecoin Core, Exodus, etc.)	`<DOGE_PAYOUT_ADDR>`
New ASC app record	App Store Connect → Apps → New iOS App. Pick a NEW bundle ID — do not reuse Satoshi Cloud Miner's.	`<BUNDLE_ID> / <ASC_APP_ID>`

ALTERNATIVE: If you do NOT have a NowPayments account, you have two alternatives. (A) **BlockCypher** + your own LTC/DOGE node — more control, more ops overhead. (B) **CoinGate** — same shape as NowPayments. The prompts below assume NowPayments; the agent will adapt if you swap.

6 · LITECOIN + DOGECOIN VARIANT

The Master Prompt (LTC + DOGE edition)

Send this as the very first message in a fresh chat. It re-uses every non-negotiable constraint from the Section 1 Master Prompt, plus the two-coin specifics.

■ PROMPT L00 MASTER PROMPT — LITECOIN + DOGECOIN

GOAL

Clone the Satoshi Cloud Miner architecture into a brand-new iOS app called <MY_APP_NAME> that supports BOTH Litecoin (LTC) and Dogecoin (DOGE). One app, both coins, single APK/IPA, coin selector in the Wallet tab. Ship to TestFlight + the App Store end-to-end.

MY CREDENTIALS (use these, do not ask later)

Apple Team ID: <TEAM_ID>
ASC API key: <ASC_KEY_ID> / <ASC_ISSUER_ID> (.p8 attached)
IAP Server API key: <IAP_KEY_ID> (.p8 attached)
EXPO_TOKEN: <EXPO_TOKEN>
ASC app record: bundle=<BUNDLE_ID> app_id=<ASC_APP_ID>
LLM key: Use the Emergent universal LLM key
NowPayments: api_key=<NOWPAY_API_KEY>
ipn_secret=<NOWPAY_IPN_SECRET>
ltc_addr=<LTC_PAYOUT_ADDR>
doge_addr=<DOGE_PAYOUT_ADDR>

COIN-SPECIFIC CONSTRAINTS (in addition to Section 1's 8 rules)

A. Live price feeds: TWO tickers — LTC/USD and DOGE/USD. Use the same 3-provider cascade as SCM (CoinGecko -> Coinbase -> Kraken). Refresh every 5 min. Expose at /api/system/rates returning {ltc_usd, doge_usd, source, fetched_at}.
B. Withdrawal: use NowPayments REST API for ALL payouts. No mock payouts. /api/diag/nowpayments must show one real successful test-mode quote round-trip before any UI is built.
C. Wallet UI: a top-of-screen segmented control (LTC | DOGE) decides which balance is shown, which yield is displayed on mining plan cards, and which address the user enters at withdrawal time.
D. IAP product IDs: identical ladder to SCM — welcome_199, rookie_299, pro_499, elite_999, ultra_1999, mega_4999, giga_9999, titan_14999, colossus_19999, adfree_399. Each plan's daily yield is stored in the DB in USD; the UI converts to LTC or DOGE using the live rate from constraint A.
E. AI Trading Agents: same 6 agents (Arbiter, Helios, Orbital, Quasar, Voltage, Sentinel). Daily commentary must reference whichever coin the user is currently viewing — pass coin=LTC|DOGE into the LLM prompt at /api/ai/agents?coin=...
F. Branding: dark theme (#0B0E14) with DUAL accent — Litecoin silver-blue (#345D9D) and Dogecoin gold (#C2A633). App icon must show BOTH glyphs over the dark background, full-bleed, pixel-verified per Section 1 constraint #3.

Everything else (Apple receipt validation, auto_ship watcher, ASC metadata flow, expo-doctor gates) is identical to Section 1. Apply all of those constraints. Confirm understanding before writing code.

6 · LITECOIN + DOGECOIN VARIANT

The 10 numbered prompts (LTC + DOGE edition)

Send these in order, one per message. They mirror Section 3's 10 prompts but with the multi-coin specifics baked in.

■ PROMPT L01 Lock in the LTC + DOGE product spec

```
Here is my PRODUCT_SPEC.md for <MY_APP_NAME>. Read every section and
confirm you understand it. Specifically confirm:
  - the 5 screens (Dashboard, Plans, Wallet, AI Agents, Profile)
  - the LTC|DOGE segmented control on the Wallet tab
  - the 10 IAP tiers (identical SKU ladder to SCM)
  - NowPayments as the withdrawal vendor with the keys above
  - the dual-rate ticker (LTC/USD + DOGE/USD)

Then push back if Apple has any policy concerns with multi-coin
payouts. Reply with a TL;DR of the spec in your own words.
```

■ PROMPT L02 Build the backend + Expo skeleton (no UI yet)

```
Build the FastAPI backend + Expo Router skeleton.
  - Auth: same as SCM (email+password JWT).
  - DB models: user.balance_ltc_sats, user.balance_doge_atomic,
    user.preferred_coin (LTC|DOGE), Machine, Transaction.
  - Stubbed endpoints (200 OK with real schema):
    /api/auth/*, /api/dashboard, /api/packages,
    /api/wallet?coin=LTC|DOGE, /api/withdraw,
    /api/system/rates, /api/ai/agents?coin=...
  - expo-router wired, placeholder Text on each tab.

Then run deep_testing_backend_v2 and paste the report. Do NOT proceed
to UI until backend regression is green.
```

■ PROMPT L03 Bootstrap App Store Connect (BEFORE any IAP code)

```
Using my ASC App Manager key, do this end-to-end via the ASC API:
  1. Verify the new app record <ASC_APP_ID> (different from SCM).
  2. Create the 10 IAPs from the spec with USD pricing:
     welcome_199, rookie_299, pro_499, elite_999, ultra_1999,
     mega_4999, giga_9999, titan_14999, colossus_19999, adfree_399.
  3. Each consumable needs a 1290x2796 review-screenshot placeholder
     (dark BG + the SKU name in neon).
  4. Move every IAP to READY_TO_SUBMIT.

Then print the final list of product IDs that exist in ASC. These are
the source of truth for SHOP_PACKAGES — no extra SKUs in the backend.
```

■ PROMPT L04 Wire the 3 live integrations — prove each is live

```
Wire these integrations in order and prove each one is live before
moving to the next:

  1. integrations/rates.py — LTC/USD + DOGE/USD via CoinGecko
     -> Coinbase -> Kraken fallback cascade. Add
     /api/diag/rates → paste the response. Confirm both rates
     are within sanity bounds (LTC 20-1000, DOGE 0.01-2).
  2. integrations/nowpayments.py — REST client with key auth.
     Add /api/diag/nowpayments → fetch /v1/status and
     /v1/min-amount?currency_from=usd&currency_to=ltc. Paste
     the response. Then do a sandbox $1 LTC payout test to
     <LTC_PAYOUT_ADDR> and paste the payout_id.
  3. integrations/apple.py — same as SCM, App Store Server API
     receipt validation. Add /api/diag/apple → use Apple's
     sandbox shared-secret test transaction.

No mocks. Each diag endpoint must return a real vendor response.
```

■ PROMPT L05 Build the UI — coin selector everywhere

```
Build every screen with the coin selector baked in.
  - Wallet tab: a segmented control at the top (LTC | DOGE).
    Selection persists in Zustand + AsyncStorage.
  - Plans tab: each card shows the daily yield converted to
    both LTC AND DOGE (small line under the USD price).
  - AI Agents tab: pass the current coin to /api/ai/agents
    so commentary references whichever coin is selected.
  - Dashboard tab: hero shows BOTH balances simultaneously.

Run yarn tsc --noEmit and npx expo-doctor after every screen. Take a
screenshot of each completed tab from the web preview and paste it.
Use real data from the diag endpoints — no Lorem Ipsum, no MOCK_*.
```

■ PROMPT L06 Marketing assets — DUAL-coin icon + verification

```
Generate all App Store marketing assets in ONE batch:

  - App icon (1024x1024, RGB, no alpha): dark background
    (#0B0E14) with the LTC glyph (silver-blue #345D9D) AND
    the DOGE glyph (gold #C2A633) side by side, full-bleed,
    no white border.
  - 12 screenshots: 1290x2796 (6.7), 1242x2688 (6.5),
    1242x2208 (5.5). At least 4 must show the LTC | DOGE
    selector clearly.
  - App Preview video: 1080x1920 H.264, 15-30s, STEREO or
    SILENT audio only (Apple rejects mono).

Run the verification pass from Section 3 Prompt 06 — sample icon edges,
PNG magic bytes, ffprobe video. Paste the report. Re-encode if any
check fails.
```

■ PROMPT L07 Harden app.json for Apple submission

```
Update /app/frontend/app.json:
  - name: "<MY_APP_NAME>" slug: "<my-app-slug>"
  - ios.bundleIdentifier: <BUNDLE_ID>
  - ios.supportsTablet: false
  - ios.config.usesNonExemptEncryption: false
  - ios.infoPlist.ITSAppUsesNonExemptEncryption: false
  - Remove every permission string unless the spec demands it
  - version: 1.0.0 buildNumber: 1

Then npx expo-doctor → must be N/N pass.
```

■ PROMPT L08 Ship to App Store Connect end-to-end (one shot)

```
Run the full ship sequence WITHOUT pausing for input:
  1. npx eas build --platform ios --profile production --non-interactive
  2. npx eas submit --platform ios --non-interactive
  3. Poll ASC builds API until processingState=VALID.
  4. Attach the build to App Store Version 1.0 via ASC API.
  5. PATCH appStoreVersionLocalizations with description, keywords,
     supportUrl, marketingUrl, promotionalText, primaryCategory.
  6. Upload all 12 screenshots + App Preview video.
  7. Create reviewSubmission, attach the version as a
     reviewSubmissionItem (IAPs auto-bundle), PATCH submitted:true.

Print the final reviewSubmission state. If anything fails, surface
Apple's exact error code — do NOT retry blindly.
```

■ PROMPT L09 Arm the auto_ship watcher for v1.0.1+

```
Create /app/backend/services/auto_ship.py for <MY_APP_NAME>:
  - 30-min APScheduler cron inside the FastAPI worker.
  - Polls ASC for version 1.0's appStoreState.
  - On READY_FOR_SALE or PENDING_APPLE_RELEASE: bump app.json,
    run eas build + eas submit, attach build to v1.0.1,
    upload fresh screenshots, submit reviewSubmission.
  - Idempotent: state in /app/store/.auto_ship_state.json so it
    never double-ships.

Confirm the watcher's first tick logged successfully and paste the
log line. Same pattern as SCM — re-use that module's code.
```

■ PROMPT L10 Final smoke test + hand-off (LTC + DOGE)

```
Run a final regression pass:
  - deep_testing_backend_v2 on all critical endpoints.
  - Take a web preview screenshot of EACH tab in BOTH coins
    (toggle LTC, screenshot; toggle DOGE, screenshot).
  - curl GET /api/system/rates and paste JSON proving live LTC
    and DOGE rates from CoinGecko/Coinbase/Kraken.
  - Show the final ASC state: version, build, IAP states,
    reviewSubmission.

Then write /app/HANDOFF.md documenting:
  - every credential (ASC, EXPO, NowPayments, LLM)
  - every cron job + its schedule
  - how to swap NowPayments for an alternative (BlockCypher
    or CoinGate) without touching UI code.
```

— end of playbook v2 —